

ECE 417/517 Sample Midterm problem bank

Instructor: Vikas Dhiman

Date: Oct 8, 2026

(1) Student name: _____ Student email: _____

About the exam

1. There are total 8 problems. You must attempt all 8.
2. Maximum marks: 75.
3. Maximum time allotted: 75 min.
4. Calculators are allowed.
5. One 8x11 in cheat sheet (both sides) is allowed.

This sample exam draws material from: *NumPy/Python intro*, *Discrete Planning*, *Probabilistic Roadmaps (PRM)*, and *3D Rotations* chapters.

Problem 1 (NumPy / broadcasting, 8 marks)

Let `alphas` be a NumPy array of shape `(N,)` holding N scalars $\alpha_1, \dots, \alpha_N$, and let `vs` be an array of shape `(N, D)` holding N vectors $\mathbf{v}_1, \dots, \mathbf{v}_N \in \mathbb{R}^D$.

a. (3 marks) Write a single NumPy expression (no explicit `for` loop) that computes $\alpha_i \mathbf{v}_i$ for every i , returning an array of shape `(N, D)`. Explain why writing `alphas * vs` directly raises a broadcasting error.

b. (3 marks) Let `As` be an array of shape `(N, M, D)` holding N matrices $A_i \in \mathbb{R}^{M \times D}$ and `bs` be an array of shape `(N, D)` holding N vectors $\mathbf{b}_i \in \mathbb{R}^D$. Write a single NumPy expression, **using only element-wise multiplication (`*`) and `.sum()`** (not `@` or `np.matmul`), that computes the batch matrix-vector product $A_i \mathbf{b}_i$ for every i , returning an array of shape `(N, M)`.

c. (2 marks) Using only `**`, `.sum()` and `np.sqrt` (i.e. **not** `np.linalg.norm`), write a single NumPy expression that computes $\|\mathbf{v}_i\|_2$ for every i in `vs`, returning an array of shape `(N,)`.

Solution:

a.

`alphas[:, np.newaxis] * vs`

`alphas` has shape $(N,)$ and `vs` has shape (N, D) . NumPy broadcasting right-aligns shapes and compares trailing dimensions: comparing $(N,)$ to (N, D) lines N up against D , which only works by coincidence when $N == D$; otherwise it raises a `ValueError`. Inserting `np.newaxis` reshapes `alphas` to $(N, 1)$, which broadcasts against (N, D) correctly (the size-1 axis is repeated along D).

b.

`(As * bs[:, np.newaxis, :]).sum(axis=-1)`

`bs[:, np.newaxis, :]` has shape $(N, 1, D)$, which broadcasts against `As` of shape (N, M, D) to give elementwise products of shape (N, M, D) ; summing over the last axis contracts the D dimension, leaving (N, M) — exactly $A_i \mathbf{b}_i$ for each i .

c.

`np.sqrt((vs**2).sum(axis=-1))`

Problem 2 (*Discrete planning abstraction and graph representations, 9 marks*)

a. (3 marks) Consider a car-like robot that can only move forward or backward along its heading, and must turn in place to move left or right. Define a suitable state space $\mathcal{S}$, action space $\mathcal{U}(\mathbf{s})$, and state transition function $f(\mathbf{s}, \mathbf{u})$ for this robot.

b. (3 marks) Convert the following undirected graph, given as an adjacency list, into its adjacency matrix:

$$G = \{ 1 : [2, 3], \quad 2 : [1, 3], \quad 3 : [1, 2, 4], \quad 4 : [3] \}$$

c. (3 marks) What is the time complexity of testing whether an edge $(i, j) \in \mathcal{E}$ exists using (i) the adjacency matrix representation and (ii) the adjacency list representation? Which representation is more space efficient for a sparse graph (where $|\mathcal{E}| \ll |\mathcal{V}|^2$), and why?

Solution:

a. State space $\mathbf{s} = (x, y, \theta) \in \mathcal{S}$, where (x, y) is position and θ is heading. Action space $\mathcal{U}(\mathbf{s}) = \{\text{forward, backward, turn_left, turn_right}\}$. Transition function, for step size d and turn increment δ :

$$f(\mathbf{s}, \mathbf{u}) = \begin{cases} (x + d \cos \theta, y + d \sin \theta, \theta) & \mathbf{u} = \text{forward} \\ (x - d \cos \theta, y - d \sin \theta, \theta) & \mathbf{u} = \text{backward} \\ (x, y, \theta + \delta) & \mathbf{u} = \text{turn_left} \\ (x, y, \theta - \delta) & \mathbf{u} = \text{turn_right} \end{cases}$$

b.

$$G_{\text{adj}} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

c. Adjacency matrix: $O(1)$, a single array lookup `G_adjacency_matrix[i][j]`.
Adjacency list: $O(\deg(i))$, since we must scan the neighbor list of i . For a sparse graph the adjacency list is more space efficient: it uses $O(|\mathcal{V}| + |\mathcal{E}|)$ space versus $O(|\mathcal{V}|^2)$ for the matrix, and when $|\mathcal{E}| \ll |\mathcal{V}|^2$ the matrix wastes space storing mostly zeros.

Problem 3 (Dijkstra and A*, 9 marks)*

Consider the weighted directed graph

$$\text{graph} = \{ s : [(a, 2), (b, 5)], a : [(b, 1), (c, 4)], b : [(c, 1)], c : [] \}$$

a. (5 marks) Run Dijkstra's algorithm starting at s . Show, step by step, which node is removed from the frontier at each iteration and the resulting `node2dist` and `node2parent` after it. Report the final shortest distance to every node and the shortest path from s to c .

b. (2 marks) What is the computational complexity of Dijkstra's algorithm in terms of $|\mathcal{V}|$ and $|\mathcal{E}|$ when the frontier is implemented with a binary-heap priority queue?

c. (2 marks) State the definition of an admissible heuristic for an optimal A-star algorithm. Explain why using $h(n) = 0$ for every node n makes A* behave identically to Dijkstra's algorithm.

Solution:

a. Step-by-step (node popped, dist so far):

Step	Node popped	dist	Frontier after processing (dist)
1	$s(0)$	dist [s] = 0	$a : 2, b : 5$
2	$a(2)$	dist [a] = 2	$b : \min(5, 2+1) = 3, c : 2+4 = 6$
3	$b(3)$	dist [b] = 3	$c : \min(6, 3+1) = 4$

Step	Node popped	dist	Frontier after processing (dist)
4	$c(4)$	dist [c] = 4	(empty)

Final `node2dist` = $\{s : 0, a : 2, b : 3, c : 4\}$, `node2parent` = $\{a : s, b : a, c : b\}$.

Shortest path $s \rightarrow c$: backtrace from c : $c \leftarrow b \leftarrow a \leftarrow s$, i.e. $s \rightarrow a \rightarrow b \rightarrow c$, with total cost $2 + 1 + 1 = 4$.

b. $O((|\mathcal{V}| + |\mathcal{E}|) \log |\mathcal{V}|)$ with a binary-heap priority queue.

c. A heuristic $h(n)$ is admissible if it never overestimates the true remaining cost to the goal, i.e. $h(n) \leq h^*(n)$ for every node n , where $h^*(n)$ is the true cost from n to the goal. $h(n) = 0$ trivially satisfies $0 \leq h^*(n)$ for any nonnegative edge costs, so it is admissible. A*'s priority is $f(n) = g(n) + h(n)$; with $h \equiv 0$ this reduces to $f(n) = g(n)$, which is exactly the priority Dijkstra uses, so the two algorithms expand nodes in the same order.

Problem 4 (Probabilistic Roadmaps, 9 marks)

Let $\mathbf{x} = (x, y)$ be a query point and let $(\mathbf{v}_s, \mathbf{v}_e)$ be an edge with start vertex $\mathbf{v}_s$ and end vertex $\mathbf{v}_e$. Parametrize the line through the edge as $\mathbf{l}(t) = \mathbf{v}_s + t(\mathbf{v}_e - \mathbf{v}_s)$, $t \in \mathbb{R}$.

a. (4 marks) Derive the value t_x at which the perpendicular from $\mathbf{x}$ meets the line $\mathbf{l}(t)$, i.e. solve $(\mathbf{l}(t_x) - \mathbf{x})^\top (\mathbf{v}_e - \mathbf{v}_s) = 0$ for t_x .

b. (2 marks) Under what condition on t_x does the closest point on the *segment* (rather than the infinite line) lie strictly in the interior of the segment, as opposed to at one of its endpoints $\mathbf{v}_s$ or $\mathbf{v}_e$?

Solution:

a. Expanding the orthogonality condition:

$$\begin{aligned}
 & (\mathbf{v}_s + (\mathbf{v}_e - \mathbf{v}_s)t_x - \mathbf{x})^\top (\mathbf{v}_e - \mathbf{v}_s) = 0 \\
 \implies & (\mathbf{v}_s - \mathbf{x})^\top (\mathbf{v}_e - \mathbf{v}_s) + (\mathbf{v}_e - \mathbf{v}_s)^\top (\mathbf{v}_e - \mathbf{v}_s) t_x = 0 \\
 \implies & t_x = \frac{(\mathbf{x} - \mathbf{v}_s)^\top (\mathbf{v}_e - \mathbf{v}_s)}{\|\mathbf{v}_e - \mathbf{v}_s\|^2}
 \end{aligned}$$

b. The closest point lies strictly inside the segment when $0 \leq t_x \leq 1$; if $t_x < 0$ the closest point is $\mathbf{v}_s$, and if $t_x > 1$ it is $\mathbf{v}_e$.

Problem 5 (Composition of homogeneous transformations, 13 marks)

A camera is mounted on a robot arm. Frame 0 is the robot base.

- Frame 1 is obtained from frame 0 by first translating 2 units along x_0 , and then rotating 90° about the (new) z -axis.
- Frame 2 (the camera frame) is obtained from frame 1 by first translating 1 unit along x_1 , and then rotating -90° about the (new) x -axis.

a. (4 marks) Write the 4×4 homogeneous transformation matrix ${}^0\mathbf{T}_1$.

b. (4 marks) Write the 4×4 homogeneous transformation matrix ${}^1\mathbf{T}_2$.

c. (5 marks) Compute ${}^0\mathbf{T}_2 = {}^0\mathbf{T}_1 {}^1\mathbf{T}_2$, the transform that takes coordinates in the camera frame to the robot base frame.

Solution:

a. "Translate then rotate" means we first move the origin by $(2, 0, 0)$ in frame 0's axes, then rotate the frame's axes about that new origin, giving

${}^0\mathbf{T}_1 = \text{Trans}(2, 0, 0) R_z(90^\circ)$:

$${}^0\mathbf{T}_1 = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ & 0 & 2 \\ \sin 90^\circ & \cos 90^\circ & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

b. Similarly ${}^1\mathbf{T}_2 = \text{Trans}(1, 0, 0) R_x(-90^\circ)$:

$${}^1\mathbf{T}_2 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & \cos(-90^\circ) & -\sin(-90^\circ) & 0 \\ 0 & \sin(-90^\circ) & \cos(-90^\circ) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

c. Multiplying the two matrices:

$${}^0\mathbf{T}_2 = {}^0\mathbf{T}_1 {}^1\mathbf{T}_2 = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & -1 & 2 \\ 1 & 0 & 0 & 1 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Problem 6 (Computational complexity and heaps, 12 marks)

Consider the following barebones BFS implementation, where the graph has $|\mathcal{V}|$ vertices and $|\mathcal{E}|$ edges:

```

def bfs_barebones(graph, start):
    seen = {start}                                # line 1
    frontier = Queue()                            # line 2
    frontier.put(start)                           # line 3
    while not frontier.empty():                   # line 4
        m = frontier.get()                        # line 5
        for neighbor in graph.get(m, []):        # line 6
            if neighbor not in seen:              # line 7
                seen.add(neighbor)                # line 8
                frontier.put(neighbor)            # line 9

```

a. (6 marks) Annotate each line (1-9) with its individual computational complexity in big-O notation, and use these to show that the overall complexity of BFS is $O(|\mathcal{V}| + |\mathcal{E}|)$.

b. (6 marks) State the max-heap property relating a node to its parent, and give the array-index formulas for `Parent(i)`, `LeftChild(i)`, `RightChild(i)` used in this course's notes for a 1-indexed array-based heap. Then state the time complexity of `Heap-Insert`, `Heap-Extract-Max`, and `Build-Heap` (heapifying an entire array of n elements) in terms of n .

Solution:

a.

```

seen = {start}                                # 0(1)
frontier = Queue()                            # 0(1)
frontier.put(start)                           # 0(1)
while not frontier.empty():                   # loop body runs once
    per vertex => 0(|V|) iterations
    m = frontier.get()                        # 0(1) per iteration
=> 0(|V|) total
    for neighbor in graph.get(m, []):        # summed over all
m, total iterations = sum of deg(m) = 0(|E|)
        if neighbor not in seen:            # 0(1) each =>
0(|E|) total
            seen.add(neighbor)                # 0(1) each
            frontier.put(neighbor)            # 0(1) each

```

The outer loop contributes $O(|\mathcal{V}|)$ (each vertex is popped from the frontier at most once), and the inner loop, summed across all iterations of the outer loop, visits each edge a constant number of times, contributing $O(|\mathcal{E}|)$. Adding these gives the overall complexity $O(|\mathcal{V}| + |\mathcal{E}|)$.

b. Max-heap property: $H[\text{Parent}(i)] \geq H[i]$ for every node i (every parent is at least as large as its children). Index formulas (1-indexed array):

$$\text{Parent}(i) = \lceil i/2 \rceil, \quad \text{LeftChild}(i) = 2i, \quad \text{RightChild}(i) = 2i + 1.$$

Runtimes: `Heap-Insert` is $O(\log n)$ (bubble up at most the height of the tree), `Heap-Extract-Max` is $O(\log n)$ (replace root with last element, then sift down), and

Build-Heap (heapifying an unsorted array of n elements bottom-up) is $O(n)$ overall, even though each individual heapify call is $O(\log n)$, because most nodes are near the bottom of the tree and require little work.

Problem 7 (Transformations from diagrams, 50 marks)



Solution

Let us find the transformation matrix

$${}^0A_1 = ?$$

Write the basis vectors of coordinate frame 1 and 0 in terms of 0:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix}$$

$$\begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix}$$

Next we include the translation:

$$\begin{bmatrix} x_0 \\ y_0 \\ z_0 \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & c+e \\ 0 & -1 & 0 & a-d \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{{}^0A_1} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix}$$

Problem 7 (Another similar problem, 25 marks)



Problem 8 (Rotation matrices, 15 marks)

- (5 marks) In your own words, write a short proof that rotation of 2D rigid body by an angle θ transforms all its points $\mathbf{x}_A$ (its local coordinate frame) to $\mathbf{x}_B$ (global coordinate frame) by the rotation matrix

$${}^BR_A = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

- (5 marks) Write the 3D rotation matrix $R_y(\beta)$ that rotates a body by angle β around the Y-axis in the Z-X plane.

Solution

2.
$$R_y(\beta) = \begin{bmatrix} \cos(\beta) & 0 & \sin(\beta) \\ 0 & 1 & 0 \\ -\sin(\beta) & 0 & \cos(\beta) \end{bmatrix}$$